

Week-12 LLM

L1 Motivation for Relative Position Encoding

- PE helps to capture the order of tokens in input seq.
- 1 approach is absolute PE.

$\overline{0} \quad \overline{1} \quad \overline{2} \quad \dots$

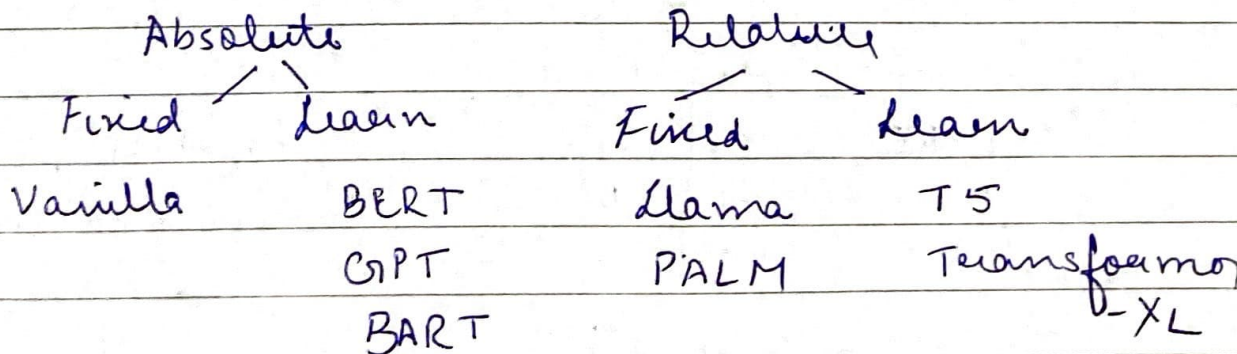
$$h_i = x_i + p_i$$

values are fixed

$$PE_{i,j} = \begin{cases} \sin\left(\frac{i}{10000^{2j/d_{\text{model}}}}\right) & j=0,2 \\ \cos & j=1,3 \end{cases}$$

- why not relative posiⁿ? currently, we are adding, what if we multiply it? currently, they are fixed, what if we learn them?

Types



- all prominent models use learned APE.
APE does learn to attend to tokens by relative posiⁿ

- We expect the model predictⁿ to be consistent for the given input seq. with (learned) absolute position encoding.
- But it was observed that the performance degrades as k increases. So, it was needed to explicitly encode the relative positions.
- The other limitaⁿ of APE is its inability to extrapolate to a longer seq. beyond the context len. of a model. Can't do length generalizaⁿ.
- Shorter training sequences to longer test ones (within the context len. of model).

L2 Relative Posiⁿ Encoding

- APE $\rightarrow$ 1 abs. posiⁿ for each token
- in RPE $\rightarrow$ we have T relative posⁿ for each token

	j	a_m
APE	0	1
RPE	0	1
	-1	0

labelled, fully-connected graph.

-
$$h_3 = x_3 + (p_{-3} + p_{-2} + \dots + p_4)$$

$$h_i = x_i + \sum_{j=0}^{T-1} p_{j-i}$$

- so if $T=1024$, we need to 1024 additions for each token. It requires T^2 addⁿ as compared to only T additions in the APE.
- $e_{ij} = q_i k_j^T$ $q_i = h_i W_Q = (x_i + p_i) W_Q$
 $k_j = h_j W_K = (x_j + p_j) W_K$

$$= (x_i + p_i) W_Q k_j^T (x_j + p_j)^T$$

$$= (x_i W_Q + p_i W_Q) (W_K^T x_j + W_K^T p_j)$$

uniformly distributed

$$= \underbrace{x_i W_Q W_K^T x_j^T}_{\text{score for } Q \& K \text{ w/o PE}} + \underbrace{x_i W_Q W_K^T p_j^T}_{\text{correlatⁿ btw. word \& posiⁿ}} + \underbrace{p_i W_Q W_K^T x_j^T}_{\text{adds a constant that comes from PE}} + \underbrace{p_i W_Q W_K^T p_j^T}_{\text{adds a constant that comes from PE}}$$

- we can inject posiⁿ info. in the attenⁿ layer directly by adjusting the pre-attenⁿ score e_{ij} .
- adding APE to the input embeddings introduce positional artefact.

$$e_{ij} = x_i W_Q (x_j W_K + p_{ij}^K)^T \quad p_{ij}^K \in \mathbb{R}^d$$

$\hookrightarrow$ posiⁿ bias.

- note now p_{ij} is d (head dim) in APE, it is d_{mod}
- But still the size of PE matrix still depends on T . Simple trick, clip anything beyond a certain k . $(2k+1)$ embeddings.

$$p_{ij}^K = W_K^K \text{clip}(j-i, k)$$

$$\text{clip}(j-i, k) = \max(-k, \min(j-i, k))$$

size of PE is $2k+1$, irrespective of T .

//_

$\alpha_{ij} = \text{softmax}(e_{ij})$
 also, add the relative posⁿ to value vector

$$z_i = \sum_j \alpha_{ij} (x_j W_v + p_{ij}^v)$$

p_{ij}^k, p_{ij}^v are shared across heads (but not across layers).

- Replacing APE by RPE with $k=16$, leads to better performance.
- Transformers $\rightarrow R_{j-i}^T$
- T5-bias $\rightarrow$ add a constant (x_{j-i}).

L3 Rotary PE (RoPE)

- $f_q(x_i, i) = x_i W_q$
 $f_k(x_j, j) = x_j W_k$ $\langle f_q(x_i, i), f_k(x_j, j) \rangle$
- we want the inner product $= g(x_i, x_j, j-i)$ btw. token embeddings to encode the relative posⁿ info.

- In APE

the book...

$p_0 \quad p_1$

read the book...

$p_1 \quad p_2$

In APE, affinity for these 2 changes, which should not happen. Irrespective of what comes before & after, the adjacent words should have the same affinity.

- somehow, we have to preserve the $\angle$ b/w. used representations so that the relative info. be encoded in the angles?

$$\langle f_q, f_k \rangle = g(x_i, x_j, j-i)$$

$$f_q(x_m, m) = x_m W_q e^{jm\theta}$$

$$f_k(x_n, n) = x_n W_k e^{jn\theta}$$

multiply a complex constant to the affine-transformed used embeddings

$$\underbrace{x, y \in \mathbb{C}^d}_{\text{complex}} \text{ is } x \cdot y^H \quad \begin{matrix} \searrow \\ \text{Hermitian} \\ \text{transpose} \end{matrix}$$

$$\langle f_q, f_k \rangle = \frac{A \rightarrow A^T \rightarrow A^H}{= x_m W_q (x_n W_q)^T e^{j(m-n)\theta \text{ conjugate}}}$$

$$e^{jm\theta} = \cos(m\theta) + j \sin(m\theta)$$

$$e^{jm\theta} = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix}$$

$\hookrightarrow$ rotation matrix R

$$x_m \in \mathbb{R}^2$$

$$f_q(x_m, m) = R W_q x_m, \text{ same for } f_k(x_n, n)$$

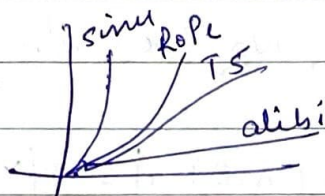
generalizes, this for $R > 2$

$$\theta = \{ \theta_i \triangleq 10000^{-\frac{2(i-1)}{d}}, i \in (1, 2, \dots, d/2) \}$$

- RoPE shows better performance. It injects at every layer of the model. It does not get added to value vectors.

L4 ALiBi & NoPE

- len gen. is the ability of a model to extrapolate to longer sequences than the one it has seen during training.
- ALiBi (attenⁿ with linear bias)
 $T = 512$ (trained) inference till $512 < T_{\text{valid}} < 16000$
 APE & TS & Sinusoidal failed but ALiBi won.

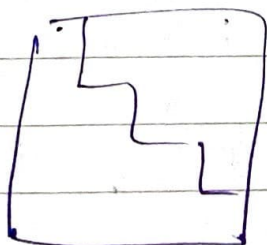


we had to modify TS.

$$e_{ij} = x_i W_Q W_K^T x_j^T + \underbrace{a_{j-i}}_{\text{learnable scalars}}$$

learnable scalars

let's hard code $a_{j-i} \Rightarrow$ as it helps to increase the inference speed.
 → just add a constant (i.e. bias) to the attenⁿ score based on its posiⁿ



$$\rightarrow \begin{bmatrix} -1 & & \\ -2 & -1 & \\ -3 & -2 & -1 \end{bmatrix} \cdot m$$

attenⁿ score

$$\text{softmax} \left(q_i k^T + \underset{\downarrow}{m_i} \cdot (-(-i-1) \dots -1, 0) \right)$$

head specific scaler

$$m_i = \frac{1}{2^i} \quad i = 1 \dots n$$

RPE, posⁿ info is added at every layer. It is cfd linear because bias value increase linearly w/ distance.

- This acts as a local windowed attenⁿ that enables ALiBi to predict the next tokens effectively.
- NOPE (don't use PE). → A causal mask implicitly encodes the posⁿ info.
- On decoder only models on other down-stream tasks, the NOPE does better performance.
 1. represents both APE & RPE.
 2. learns to use RPE in practice.
- remember, APE has to learnt initially, but the RPE goes in every layer.